DevOps Project Report: Node.js REST API

Mariem Charef

January 2025

1 Project Overview

This project implements a production-ready REST API using **Node.js and Express**, designed to demonstrate modern **DevOps practices**. The application provides CRUD operations for users and posts while focusing on containerization, orchestration, observability, security, and CI/CD automation.

The primary goal is to build a scalable and maintainable backend service that can be deployed consistently across environments using cloud-native technologies.

2 System Architecture

The architecture follows a layered and modular design:

- **Client Layer**: Sends HTTP requests to the API.
- **Application Layer**: Express.js handles routing, validation, logging, and error handling.
- **Data Layer**: MongoDB is used for persistent storage of users and posts.

The application is containerized using Docker and deployed on Kubernetes, allowing horizontal scalability and fault isolation.

3 Tools and Technologies

The main tools and technologies used in this project are:

- **Backend**: Node.js 20, Express.js
- **Database**: MongoDB
- **Containerization**: Docker, Docker Compose
- **Orchestration**: Kubernetes (minikube / kind)
- **Testing**: Jest with MongoDB Memory Server
- **CI/CD**: GitHub Actions

These tools ensure portability, automation, and reliability throughout the application lifecycle.

4 Observability

The project implements the three pillars of observability:

- **Logging**: Structured JSON logs for all requests and errors.
- **Metrics**: Prometheus metrics exposed via a `/metrics` endpoint.
- **Tracing**: Unique trace IDs assigned to each request for log correlation.

This observability stack enables effective monitoring, debugging, and performance analysis in both development and production environments.

5 Security

Several security best practices are applied:

- Password hashing using **bcryptjs**
- Secure HTTP headers to mitigate common web vulnerabilities
- Environment variables for managing sensitive configuration
- Static and Dynamic Application Security Testing (SAST and DAST)

These measures reduce the attack surface and help protect user data.

6 Kubernetes Setup

The Kubernetes setup includes:

- A deployment for the Node.js API with configurable replicas
- A service to expose the application internally or externally
- A MongoDB deployment for data persistence

Kubernetes enables rolling updates, scaling, and reliable service management, making the application cloud-ready.

7 Lessons Learned

Through this project, several key lessons were learned:

- Containerization ensures environment consistency and simplifies deployments
- Observability is essential for diagnosing issues in distributed systems
- Automating CI/CD pipelines improves reliability and development speed
- Security must be integrated from the beginning, not added later
- Kubernetes provides powerful abstractions but requires careful configuration

8 Conclusion

This project successfully demonstrates an end-to-end DevOps workflow for a modern backend application. By combining containerization, Kubernetes orchestration, observability, security, and CI/CD automation, the system achieves scalability, reliability, and maintainability suitable for production environments.